EcoPick - Backend API Requirements

1. Tổng quan kiến trúc Backend

Tech Stack đề xuất

Layer	Technology	Lý do
Runtime	Node.js 20 LTS	Ecosystem lớn, phù hợp real-time
Framework	NestJS hoặc Express.js	Structured, TypeScript native
Database	PostgreSQL 16 + PostGIS	Relational + geospatial queries
Cache	Redis	Session, leaderboard, rate limiting
Object Storage	AWS S3 / Cloudflare R2	Ảnh, video uploads
Auth	JWT + OAuth 2.0	Stateless, social login
Real-time	Socket.IO / WebSocket	Chat, notifications
Push	Firebase Cloud Messaging (FCM)	Cross-platform push notifications
Search	PostgreSQL Full-text / Meilisearch	Search posts, events, items
Queue	BullMQ (Redis)	Background jobs (image processing, notifications)
CDN	Cloudflare	Image delivery, caching
Hosting	Railway / Render / AWS ECS	Containerized deployment

Architecture Diagram

```
┌──────────────────────────────────────────────────────────────────┐
│                        Mobile App (Expo)                         │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐  ┌──────────────┐     │
│  │ REST API │  │ WebSocket│  │   Push   │  │  S3 Direct   │     │
│  │ (HTTPS)  │  │  (WSS)   │  │  (FCM)   │  │  Upload      │     │
│  └────┬─────┘  └────┬─────┘  └────┬─────┘  └──────┬───────┘     │
└───────┼─────────────┼────────────┼───────────────┼──────────────┘
        │             │            │               │
┌───────▼─────────────▼────────────▼───────────────▼──────────────┐
│                    API Gateway / Load Balancer                   │
│                    (Rate Limiting, CORS, Auth)                   │
└───────┬─────────────┬────────────┬───────────────┬──────────────┘
        │             │            │               │
┌───────▼──────┐  ┌───▼──────┐  ┌──▼───────┐  ┌────▼──────────┐
│  API Server  │  │ WebSocket│  │  Worker  │  │  S3 / R2      │
│  (NestJS)    │  │  Server  │  │  Queue   │  │  Storage      │
```

```
│              │ │(Socket.IO)│ │(BullMQ)  │ │              │
└──────────────┘ └──────────┘ └──────────┘ └──────────────┘
        │             │          │
┌───────▼─────────────▼──────────▼──────────────────────────┐
│                     Data Layer                             │
│  ┌──────────────┐  ┌──────────┐  ┌──────────────────┐    │
│  │ PostgreSQL   │  │  Redis   │  │ Meilisearch      │    │
│  │ + PostGIS    │  │  Cache   │  │ (Search Engine)  │    │
│  └──────────────┘  └──────────┘  └──────────────────┘    │
└────────────────────────────────────────────────────────────┘
```

---

2. Database Schema

2.1 Users

```
CREATE TABLE users (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email           VARCHAR(255) UNIQUE NOT NULL,
  password_hash   VARCHAR(255), -- NULL for OAuth-only users
  full_name       VARCHAR(100) NOT NULL,
  username        VARCHAR(50) UNIQUE NOT NULL,
  bio             TEXT DEFAULT '',
  avatar_url      VARCHAR(500),
  role            VARCHAR(20) DEFAULT 'user', -- user, organizer, moderator,
admin
  level           INT DEFAULT 1,
  eco_points      INT DEFAULT 0,
  total_reports   INT DEFAULT 0,
  total_events    INT DEFAULT 0,
  total_trees     INT DEFAULT 0,
  follower_count  INT DEFAULT 0,
  following_count INT DEFAULT 0,
  auth_provider   VARCHAR(20) DEFAULT 'email', -- email, google, apple
  auth_provider_id VARCHAR(255),
  fcm_token       VARCHAR(500),
  is_verified     BOOLEAN DEFAULT FALSE,
  is_active       BOOLEAN DEFAULT TRUE,
  last_login_at   TIMESTAMPTZ,
  created_at      TIMESTAMPTZ DEFAULT NOW(),
  updated_at      TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_users_email ON users(email);
CREATE INDEX idx_users_username ON users(username);
CREATE INDEX idx_users_eco_points ON users(eco_points DESC);
```

2.2 Waste Reports

```sql
CREATE TABLE waste_reports (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         UUID NOT NULL REFERENCES users(id),
  latitude        DOUBLE PRECISION NOT NULL,
  longitude       DOUBLE PRECISION NOT NULL,
  location        GEOGRAPHY(Point, 4326) NOT NULL, -- PostGIS
  address         VARCHAR(500),
  city            VARCHAR(100),
  country         VARCHAR(100),
  severity        VARCHAR(20) NOT NULL, -- light, medium, heavy, extreme
  waste_types     TEXT[] NOT NULL, -- {plastic, paper, glass, organic,
metal}
  description     TEXT,
  status          VARCHAR(20) DEFAULT 'reported', -- reported, verified,
in_progress, cleaned
  upvote_count    INT DEFAULT 0,
  comment_count   INT DEFAULT 0,
  cleaned_by      UUID REFERENCES users(id),
  cleaned_at      TIMESTAMPTZ,
  created_at      TIMESTAMPTZ DEFAULT NOW(),
  updated_at      TIMESTAMPTZ DEFAULT NOW()
);

-- Spatial index for nearby queries
CREATE INDEX idx_reports_location ON waste_reports USING GIST(location);
CREATE INDEX idx_reports_status ON waste_reports(status);
CREATE INDEX idx_reports_severity ON waste_reports(severity);
CREATE INDEX idx_reports_user ON waste_reports(user_id);
CREATE INDEX idx_reports_created ON waste_reports(created_at DESC);
```

2.3 Report Photos

```sql
CREATE TABLE report_photos (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  report_id   UUID NOT NULL REFERENCES waste_reports(id) ON DELETE CASCADE,
  url         VARCHAR(500) NOT NULL,
  thumbnail_url VARCHAR(500),
  width       INT,
  height      INT,
  order_index INT DEFAULT 0,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_report_photos_report ON report_photos(report_id);
```

2.4 Events

```sql
CREATE TABLE events (
  id                  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
```

```
  organizer_id        UUID NOT NULL REFERENCES users(id),
  title               VARCHAR(200) NOT NULL,
  description         TEXT NOT NULL,
  cover_image_url     VARCHAR(500),
  type                VARCHAR(30) NOT NULL, -- cleanup, tree_planting,
beach_cleanup, education, tour
  start_date          TIMESTAMPTZ NOT NULL,
  end_date            TIMESTAMPTZ NOT NULL,
  latitude            DOUBLE PRECISION NOT NULL,
  longitude           DOUBLE PRECISION NOT NULL,
  location            GEOGRAPHY(Point, 4326) NOT NULL,
  address             VARCHAR(500) NOT NULL,
  max_participants    INT DEFAULT 50,
  current_participants INT DEFAULT 0,
  equipment           TEXT[], -- {"Gloves", "Trash bags", "Water bottle"}
  difficulty          VARCHAR(10) DEFAULT 'easy', -- easy, medium, hard
  eco_points_reward   INT DEFAULT 100,
  status              VARCHAR(20) DEFAULT 'upcoming', -- upcoming, ongoing,
completed, cancelled
  created_at          TIMESTAMPTZ DEFAULT NOW(),
  updated_at          TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_events_location ON events USING GIST(location);
CREATE INDEX idx_events_start ON events(start_date);
CREATE INDEX idx_events_status ON events(status);
CREATE INDEX idx_events_organizer ON events(organizer_id);
```

2.5 Event Participants

```
CREATE TABLE event_participants (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id    UUID NOT NULL REFERENCES events(id) ON DELETE CASCADE,
  user_id     UUID NOT NULL REFERENCES users(id),
  status      VARCHAR(20) DEFAULT 'joined', -- joined, checked_in,
completed, left
  joined_at   TIMESTAMPTZ DEFAULT NOW(),
  checked_in_at TIMESTAMPTZ,
  UNIQUE(event_id, user_id)
);

CREATE INDEX idx_event_participants_event ON event_participants(event_id);
CREATE INDEX idx_event_participants_user ON event_participants(user_id);
```

2.6 Tour Stops (cho Tour events)

```
CREATE TABLE tour_stops (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  event_id    UUID NOT NULL REFERENCES events(id) ON DELETE CASCADE,
  name        VARCHAR(200) NOT NULL,
```

```
  description TEXT,
  latitude    DOUBLE PRECISION NOT NULL,
  longitude   DOUBLE PRECISION NOT NULL,
  location    GEOGRAPHY(Point, 4326) NOT NULL,
  order_index INT NOT NULL,
  goal        VARCHAR(500), -- "Collect 50kg plastic waste"
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_tour_stops_event ON tour_stops(event_id);
```

2.7 Posts (Social)

```
CREATE TABLE posts (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  author_id     UUID NOT NULL REFERENCES users(id),
  content       TEXT NOT NULL,
  location_name VARCHAR(200),
  latitude      DOUBLE PRECISION,
  longitude     DOUBLE PRECISION,
  hashtags      TEXT[],
  tagged_users  UUID[],
  like_count    INT DEFAULT 0,
  comment_count INT DEFAULT 0,
  share_count   INT DEFAULT 0,
  related_report_id UUID REFERENCES waste_reports(id),
  related_event_id  UUID REFERENCES events(id),
  is_hidden     BOOLEAN DEFAULT FALSE,
  created_at    TIMESTAMPTZ DEFAULT NOW(),
  updated_at    TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_posts_author ON posts(author_id);
CREATE INDEX idx_posts_created ON posts(created_at DESC);
CREATE INDEX idx_posts_hashtags ON posts USING GIN(hashtags);
```

2.8 Post Media

```
CREATE TABLE post_media (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  post_id     UUID NOT NULL REFERENCES posts(id) ON DELETE CASCADE,
  url         VARCHAR(500) NOT NULL,
  thumbnail_url VARCHAR(500),
  type        VARCHAR(10) NOT NULL, -- image, video
  width       INT,
  height      INT,
  order_index INT DEFAULT 0,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

2.9 Comments (polymorphic — cho reports, posts, events)

```
CREATE TABLE comments (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id       UUID NOT NULL REFERENCES users(id),
  -- Polymorphic: chỉ 1 trong 3 cái NOT NULL
  report_id     UUID REFERENCES waste_reports(id) ON DELETE CASCADE,
  post_id       UUID REFERENCES posts(id) ON DELETE CASCADE,
  event_id      UUID REFERENCES events(id) ON DELETE CASCADE,
  content       TEXT NOT NULL,
  photo_url     VARCHAR(500),
  created_at    TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_comments_report ON comments(report_id) WHERE report_id IS
NOT NULL;
CREATE INDEX idx_comments_post ON comments(post_id) WHERE post_id IS NOT
NULL;
```

2.10 Likes

```
CREATE TABLE likes (
  id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id   UUID NOT NULL REFERENCES users(id),
  post_id   UUID REFERENCES posts(id) ON DELETE CASCADE,
  report_id UUID REFERENCES waste_reports(id) ON DELETE CASCADE,
  created_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(user_id, post_id),
  UNIQUE(user_id, report_id)
);
```

2.11 Follows

```
CREATE TABLE follows (
  id           UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  follower_id  UUID NOT NULL REFERENCES users(id),
  following_id UUID NOT NULL REFERENCES users(id),
  created_at   TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(follower_id, following_id),
  CHECK(follower_id != following_id)
);

CREATE INDEX idx_follows_follower ON follows(follower_id);
CREATE INDEX idx_follows_following ON follows(following_id);
```

2.12 Badges

```
```

```sql
CREATE TABLE badges (
  id          VARCHAR(50) PRIMARY KEY, -- 'first_report', 'cleanup_rookie'
  name        VARCHAR(100) NOT NULL,
  description TEXT NOT NULL,
  icon_url    VARCHAR(500),
  category    VARCHAR(30), -- reporting, events, social, streaks, exchange
  requirement TEXT NOT NULL -- JSON: {"type": "report_count", "value": 50}
);

CREATE TABLE user_badges (
  id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id   UUID NOT NULL REFERENCES users(id),
  badge_id  VARCHAR(50) NOT NULL REFERENCES badges(id),
  earned_at TIMESTAMPTZ DEFAULT NOW(),
  UNIQUE(user_id, badge_id)
);
```

2.13 Exchange Items (Marketplace)

```sql
CREATE TABLE exchange_items (
  id            UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  owner_id      UUID NOT NULL REFERENCES users(id),
  title         VARCHAR(200) NOT NULL,
  description   TEXT,
  category      VARCHAR(30) NOT NULL, -- clothing, books, electronics,
furniture, kitchen, sports, other
  condition     VARCHAR(20) NOT NULL, -- new, like_new, good, fair
  type          VARCHAR(10) NOT NULL, -- give, exchange
  exchange_for  VARCHAR(200), -- "Looking for a book bag"
  latitude      DOUBLE PRECISION,
  longitude     DOUBLE PRECISION,
  location      GEOGRAPHY(Point, 4326),
  address       VARCHAR(500),
  status        VARCHAR(20) DEFAULT 'available', -- available, reserved,
completed, removed
  requests_count INT DEFAULT 0,
  created_at    TIMESTAMPTZ DEFAULT NOW(),
  updated_at    TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_exchange_location ON exchange_items USING GIST(location);
CREATE INDEX idx_exchange_category ON exchange_items(category);
CREATE INDEX idx_exchange_status ON exchange_items(status);
```

2.14 Notifications

```sql
CREATE TABLE notifications (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id),
```

```
  type        VARCHAR(30) NOT NULL,
  title       VARCHAR(200) NOT NULL,
  body        TEXT NOT NULL,
  data        JSONB, -- {"reportId": "xxx", "screen": "location"}
  is_read     BOOLEAN DEFAULT FALSE,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_notifications_user ON notifications(user_id, created_at
DESC);
CREATE INDEX idx_notifications_unread ON notifications(user_id) WHERE
is_read = FALSE;
```

2.15 Chat

```
CREATE TABLE conversations (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  type        VARCHAR(10) NOT NULL, -- direct, group
  event_id    UUID REFERENCES events(id), -- NULL for direct chats
  name        VARCHAR(200), -- for group chats
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE TABLE conversation_members (
  conversation_id UUID NOT NULL REFERENCES conversations(id) ON DELETE
CASCADE,
  user_id         UUID NOT NULL REFERENCES users(id),
  joined_at       TIMESTAMPTZ DEFAULT NOW(),
  last_read_at    TIMESTAMPTZ,
  PRIMARY KEY(conversation_id, user_id)
);

CREATE TABLE messages (
  id              UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  conversation_id UUID NOT NULL REFERENCES conversations(id) ON DELETE
CASCADE,
  sender_id       UUID NOT NULL REFERENCES users(id),
  content         TEXT,
  photo_url       VARCHAR(500),
  location_lat    DOUBLE PRECISION,
  location_lng    DOUBLE PRECISION,
  created_at      TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_messages_conversation ON messages(conversation_id,
created_at DESC);
```

2.16 Points Log

```
```

```sql
CREATE TABLE points_log (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id     UUID NOT NULL REFERENCES users(id),
  points      INT NOT NULL, -- positive = earn, negative = spend
  reason      VARCHAR(50) NOT NULL, -- report_created, event_completed,
reward_redeemed...
  reference_id UUID, -- ID of the related entity
  created_at  TIMESTAMPTZ DEFAULT NOW()
);

CREATE INDEX idx_points_user ON points_log(user_id, created_at DESC);
```

---

3. API Endpoints

3.1 Authentication

```
POST   /api/v1/auth/register          Đăng ký email
POST   /api/v1/auth/login             Đăng nhập email
POST   /api/v1/auth/google            Đăng nhập Google OAuth
POST   /api/v1/auth/apple             Đăng nhập Apple
POST   /api/v1/auth/verify-email      Verify OTP email
POST   /api/v1/auth/forgot-password   Gửi email reset password
POST   /api/v1/auth/reset-password    Reset password bằng token
POST   /api/v1/auth/refresh           Refresh JWT token
POST   /api/v1/auth/logout            Logout (invalidate refresh token)
```

`POST /api/v1/auth/register`

```json
// Request
{
  "email": "user@example.com",
  "password": "SecureP@ss1",
  "fullName": "Nguyen Van A",
  "username": "nguyenvana"
}

// Response 201
{
  "success": true,
  "data": {
    "user": { "id": "uuid", "email": "...", "fullName": "...", "username":
"..." },
    "message": "Verification email sent"
  }
}

// Error 409
{
```

```
  "success": false,
  "error": { "code": "EMAIL_EXISTS", "message": "Email already registered" }
}
```

POST /api/v1/auth/login

```
// Request
{
  "email": "user@example.com",
  "password": "SecureP@ss1"
}

// Response 200
{
  "success": true,
  "data": {
    "user": { "id": "uuid", "email": "...", "fullName": "...", "level": 3,
"ecoPoints": 1250 },
    "accessToken": "eyJhbG...",
    "refreshToken": "dGhpcyBpcyBh..."
  }
}
```

3.2 User Profile

```
GET    /api/v1/users/me                Lấy profile hiện tại
PATCH  /api/v1/users/me                Cập nhật profile
DELETE /api/v1/users/me                Xóa tài khoản (soft delete)
GET    /api/v1/users/:id               Xem profile user khác
GET    /api/v1/users/:id/stats         Thống kê user
GET    /api/v1/users/:id/activity      Lịch sử hoạt động
GET    /api/v1/users/:id/badges        Huy hiệu của user
POST   /api/v1/users/me/avatar         Upload avatar
POST   /api/v1/users/me/fcm-token      Cập nhật FCM token
```

GET /api/v1/users/me

```
// Response 200
{
  "success": true,
  "data": {
    "id": "uuid",
    "email": "user@example.com",
    "fullName": "Nguyen Van A",
    "username": "nguyenvana",
    "bio": "Eco warrior from HCMC 🌿",
    "avatarUrl": "https://cdn.ecopick.vn/avatars/uuid.jpg",
    "role": "user",
```

```
    "level": 3,
    "ecoPoints": 1250,
    "stats": {
      "totalReports": 15,
      "totalEvents": 8,
      "treesPlanted": 12,
      "followerCount": 42,
      "followingCount": 28
    },
    "createdAt": "2024-01-15T08:00:00Z"
  }
}
```

3.3 Waste Reports

```
POST   /api/v1/reports                  Tạo report mới
GET    /api/v1/reports                  Lấy danh sách (filter, pagination)
GET    /api/v1/reports/nearby           Lấy reports gần vị trí (geo query)
GET    /api/v1/reports/:id              Chi tiết report
PATCH  /api/v1/reports/:id/status       Cập nhật status
POST   /api/v1/reports/:id/upvote       Upvote report
DELETE /api/v1/reports/:id/upvote       Remove upvote
POST   /api/v1/reports/:id/claim        Nhận task dọn dẹp
GET    /api/v1/reports/:id/comments     Lấy comments
POST   /api/v1/reports/:id/comments     Thêm comment
POST   /api/v1/uploads/presigned        Lấy presigned URL để upload ảnh
```

`GET /api/v1/reports/nearby`

```
// Query params
// ?
lat=10.7769&lng=106.7009&radius=5000&severity=heavy,extreme&status=reported&
page=1&limit=50

// Response 200
{
  "success": true,
  "data": {
    "reports": [
      {
        "id": "uuid",
        "latitude": 10.7769,
        "longitude": 106.7009,
        "address": "123 Nguyen Hue, D1",
        "severity": "heavy",
        "wasteTypes": ["plastic", "metal"],
        "photoUrls": ["https://cdn.ecopick.vn/reports/photo1.jpg"],
        "status": "reported",
        "upvoteCount": 5,
        "commentCount": 2,
```

```
        "distance": 1.2, // km from query point
        "reportedBy": {
          "id": "uuid",
          "fullName": "Tran B",
          "avatarUrl": "https://..."
        },
        "createdAt": "2024-06-15T14:30:00Z"
      }
    ],
    "pagination": {
      "page": 1,
      "limit": 50,
      "total": 127,
      "totalPages": 3
    }
  }
}
```

POST /api/v1/reports

```
// Request (multipart/form-data or JSON with presigned URLs)
{
  "latitude": 10.7769,
  "longitude": 106.7009,
  "severity": "heavy",
  "wasteTypes": ["plastic", "glass"],
  "description": "Large pile of plastic bottles near the park",
  "photoKeys": ["uploads/reports/abc123.jpg"] // S3 keys from presigned
upload
}

// Response 201
{
  "success": true,
  "data": {
    "report": { "id": "uuid", ... },
    "pointsEarned": 50,
    "newTotal": 1300,
    "badgeUnlocked": null // or badge object if milestone reached
  }
}
```

3.4 Events

```
GET    /api/v1/events                  Danh sách events (filter, pagination)
GET    /api/v1/events/nearby           Events gần vị trí
GET    /api/v1/events/:id              Chi tiết event
POST   /api/v1/events                  Tạo event (organizer+)
PATCH  /api/v1/events/:id              Cập nhật event
DELETE /api/v1/events/:id              Hủy event
```

```
POST   /api/v1/events/:id/join         Tham gia event
DELETE /api/v1/events/:id/join         Rời event
POST   /api/v1/events/:id/check-in    Check-in tại event
GET    /api/v1/events/:id/participants Danh sách participants
GET    /api/v1/events/:id/chat         Lấy tin nhắn chat nhóm
```

GET /api/v1/events

```
// Query: ?status=upcoming&type=cleanup&lat=10.77&lng=106.70&radius=10000&page=1&limit=20

// Response 200
{
  "success": true,
  "data": {
    "events": [
      {
        "id": "uuid",
        "title": "Beach Cleanup - Vung Tau",
        "type": "beach_cleanup",
        "coverImageUrl": "https://cdn.ecopick.vn/events/cover1.jpg",
        "startDate": "2024-07-20T08:00:00Z",
        "endDate": "2024-07-20T12:00:00Z",
        "address": "Back Beach, Vung Tau",
        "maxParticipants": 30,
        "currentParticipants": 18,
        "difficulty": "easy",
        "ecoPointsReward": 200,
        "status": "upcoming",
        "distance": 4.5,
        "organizer": { "id": "uuid", "fullName": "Green HCMC", "avatarUrl": "..." },
        "participantAvatars": ["url1", "url2", "url3"]
      }
    ],
    "pagination": { "page": 1, "limit": 20, "total": 8 }
  }
}
```

3.5 Tours

```
GET    /api/v1/tours/:id                Chi tiết tour (event + stops)
GET    /api/v1/tours/:id/stops          Danh sách stops
POST   /api/v1/tours/:id/stops/:stopId/check-in   Check-in tại stop
```

3.6 Social / Posts

```
```

```
GET    /api/v1/feed                    Newsfeed (following)
GET    /api/v1/feed/discover           Trending / discover
POST   /api/v1/posts                   Tạo post
GET    /api/v1/posts/:id               Chi tiết post
DELETE /api/v1/posts/:id               Xóa post
POST   /api/v1/posts/:id/like          Like post
DELETE /api/v1/posts/:id/like          Unlike post
GET    /api/v1/posts/:id/comments      Comments của post
POST   /api/v1/posts/:id/comments      Thêm comment
POST   /api/v1/users/:id/follow        Follow user
DELETE /api/v1/users/:id/follow        Unfollow user
GET    /api/v1/users/:id/followers     Danh sách followers
GET    /api/v1/users/:id/following     Danh sách following
GET    /api/v1/hashtags/trending       Trending hashtags
```

3.7 Chat / Messaging

```
GET    /api/v1/conversations           Danh sách conversations
GET    /api/v1/conversations/:id       Chi tiết conversation + messages
POST   /api/v1/conversations           Tạo conversation (DM)
POST   /api/v1/conversations/:id/messages   Gửi message (cũng qua WebSocket)

// WebSocket events
ws://api.ecopick.vn/ws
  → join_conversation(conversationId)
  → send_message({ conversationId, content, photoUrl? })
  ← new_message({ id, senderId, content, createdAt })
  ← typing({ conversationId, userId })
  ← read_receipt({ conversationId, userId, lastReadAt })
```

3.8 Exchange / Marketplace

```
GET    /api/v1/exchange                Danh sách items
GET    /api/v1/exchange/nearby         Items gần vị trí
GET    /api/v1/exchange/:id            Chi tiết item
POST   /api/v1/exchange                Đăng item
PATCH  /api/v1/exchange/:id            Cập nhật item
DELETE /api/v1/exchange/:id            Gỡ item
POST   /api/v1/exchange/:id/request    Gửi request nhận/đổi
PATCH  /api/v1/exchange/:id/request/:reqId   Approve/reject request
POST   /api/v1/exchange/:id/complete   Đánh dấu hoàn thành giao dịch
```

3.9 Gamification

```
GET    /api/v1/badges                  Tất cả badges (earned + locked)
GET    /api/v1/leaderboard             Bảng xếp hạng
GET    /api/v1/leaderboard?period=weekly&scope=city   Lọc theo period/scope
GET    /api/v1/rewards                 Danh sách rewards
```

```
POST   /api/v1/rewards/:id/redeem      Đổi reward
GET    /api/v1/points/history          Lịch sử điểm
GET    /api/v1/challenges/daily        Daily challenges
```

3.10 Notifications

```
GET    /api/v1/notifications           Danh sách notifications (paginated)
PATCH  /api/v1/notifications/:id/read  Đánh dấu đã đọc
PATCH  /api/v1/notifications/read-all  Đánh dấu tất cả đã đọc
GET    /api/v1/notifications/unread-count  Số notification chưa đọc
```

3.11 Upload

```
POST   /api/v1/uploads/presigned       Lấy presigned URL
```

```json
// Request
{
  "fileType": "image/jpeg",
  "category": "report" // report, event, avatar, post, exchange
}

// Response 200
{
  "success": true,
  "data": {
    "uploadUrl": "https://s3.amazonaws.com/ecopick-uploads/...",
    "fileKey": "uploads/reports/abc123.jpg",
    "publicUrl": "https://cdn.ecopick.vn/reports/abc123.jpg",
    "expiresIn": 300 // seconds
  }
}
```

---

4. Authentication Flow

JWT Token Strategy

```
Access Token:
  - Lifetime: 15 minutes
  - Payload: { userId, role, email }
  - Sent in: Authorization: Bearer <token>

Refresh Token:
  - Lifetime: 30 days
  - Stored in: Redis (server-side) + SecureStore (client-side)
  - Rotation: Mỗi lần refresh → invalidate token cũ, cấp token mới
```

Auth Middleware

```
1. Mọi request → check Authorization header
2. Verify JWT signature + expiration
3. Extract userId → attach to req.user
4. Role-based check cho protected routes (organizer, admin)
```

---

5. Rate Limiting

Endpoint group	Limit	Window
Auth (login/register)	5 requests	15 min
Report creation	10 requests	24h
Upload	20 requests	1h
General API	100 requests	1 min
Search	30 requests	1 min

---

6. Background Jobs (Queue)

Job	Trigger	Action
`process_image`	Sau upload	Resize, tạo thumbnail, optimize
`send_push`	Notification created	Gửi FCM push notification
`send_email`	Register, forgot password	Gửi email qua SMTP
`check_badges`	Points changed	Kiểm tra unlock badge mới
`update_leaderboard`	Points changed	Cập nhật Redis sorted set
`geocode_address`	Report/event created	Reverse geocode lat/lng → address
`event_reminders`	Cron 24h trước event	Push reminder to participants
`cleanup_expired`	Daily cron	Xóa unverified accounts, expired tokens

---

7. Error Response Format

```json
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Invalid input data",
    "details": [
      { "field": "email", "message": "Invalid email format" },
      { "field": "password", "message": "Must be at least 8 characters" }
```

```
      ]
    }
  }
```

Error Codes

Code	HTTP Status	Mô tả
`VALIDATION_ERROR`	400	Input không hợp lệ
`UNAUTHORIZED`	401	Chưa đăng nhập / token hết hạn
`FORBIDDEN`	403	Không có quyền
`NOT_FOUND`	404	Resource không tồn tại
`CONFLICT`	409	Duplicate (email exists, already joined...)
`RATE_LIMITED`	429	Quá nhiều request
`INTERNAL_ERROR`	500	Lỗi server